

Deep Learning Recap

Instructor

Dr. Mayank Baranwal

Senior Scientist, TCS Research



Deep Learning: Hype or Reality



Andrew Ng

I have worked all my life in Machine Learning, and I've never seen one algorithm knock over benchmarks like Deep Learning.



Geoffrey Hinton

Deep Learning is an algorithm which has no theoretical limitations of what it can learn; the more data you give and the more computational time you provide, the better it is.



Demis Hassabis

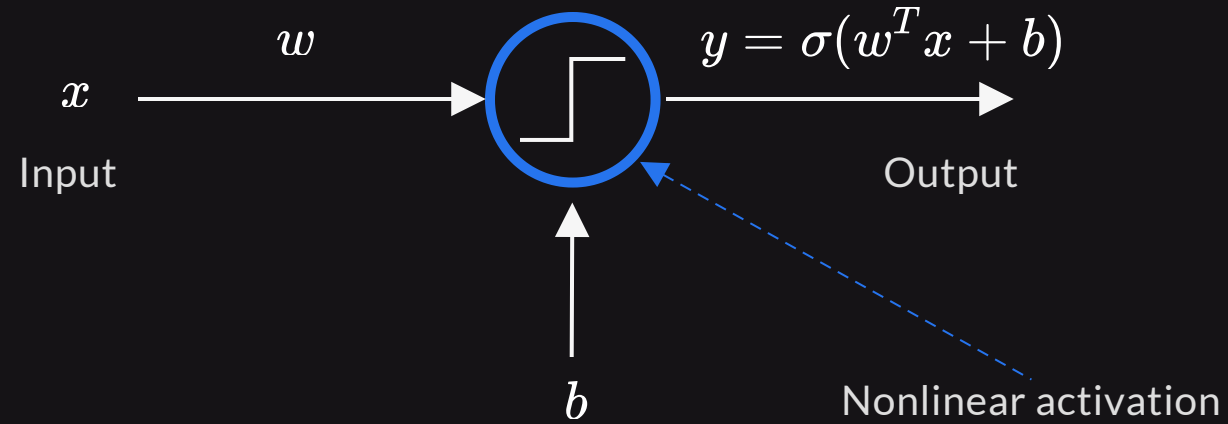
For a very-long time it will be a complementary tool that human scientists and experts can use to help them with the things that humans are not naturally good at.



Dileep George

Human-level artificial intelligence has the potential to help humanity thrive more than any invention that has come before it.

Single Neuron



$$y = \begin{cases} 1 & \text{if } w^T x + b \geq 0 \\ 0 & \text{else} \end{cases}$$

Function Approximation: Example XOR Gate

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

How can we model this using perceptron?

Function Approximation: Example XOR Gate

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

How can we model this using perceptron?

We can model this using linear inequalities:

Output 1 if $1 \leq x_1 + x_2 < 2$

Function Approximation: Example XOR Gate

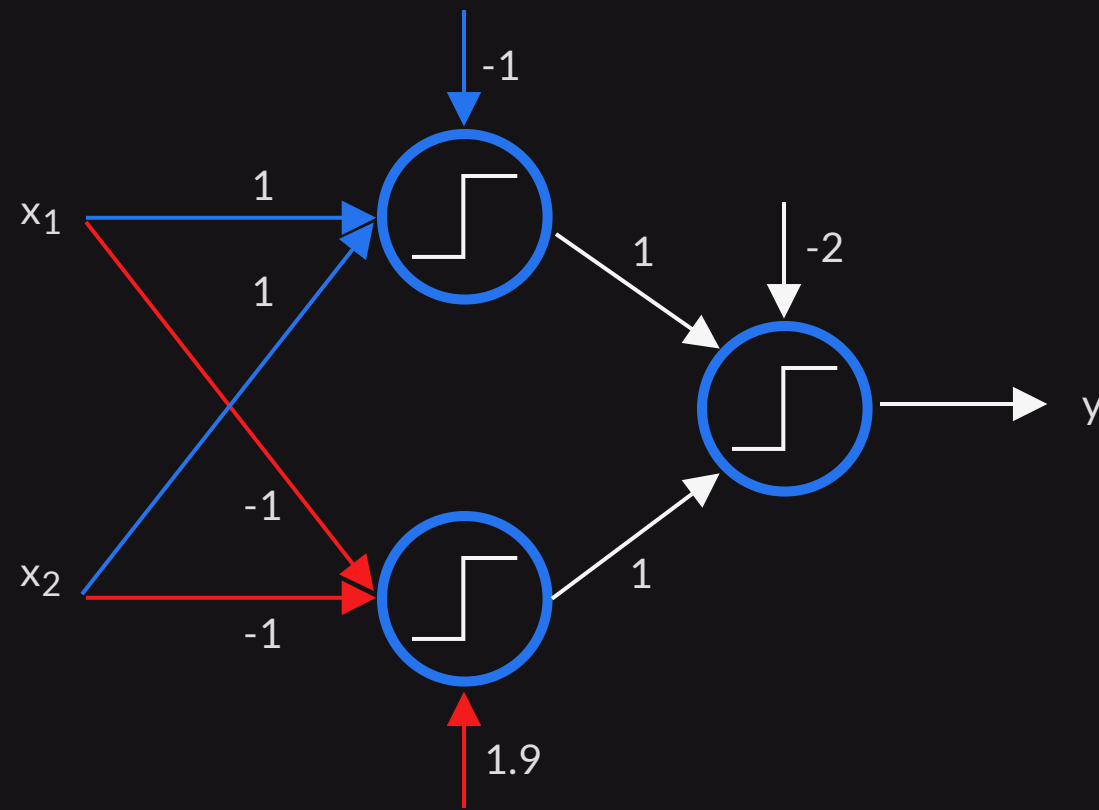
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

How can we model this using perceptron?

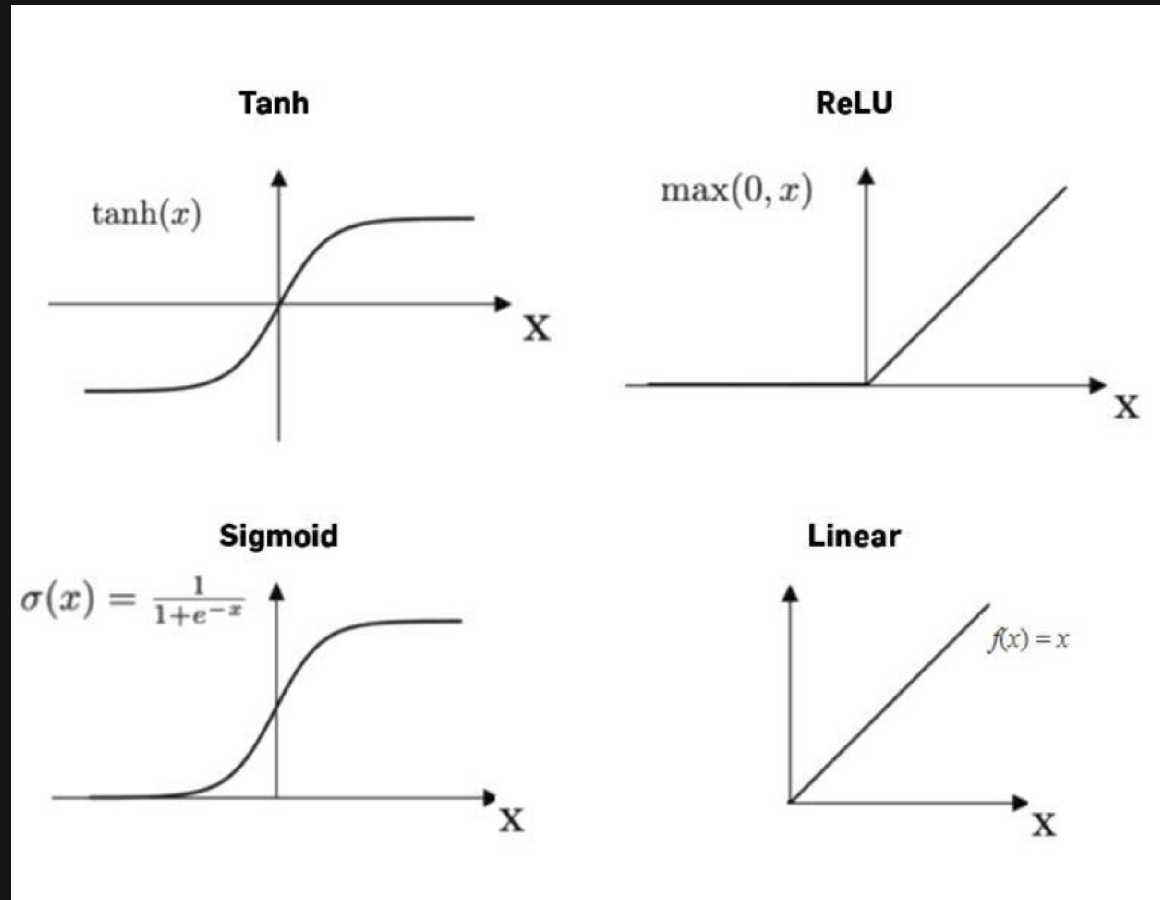
We can model this using linear inequalities:

Output 1 if

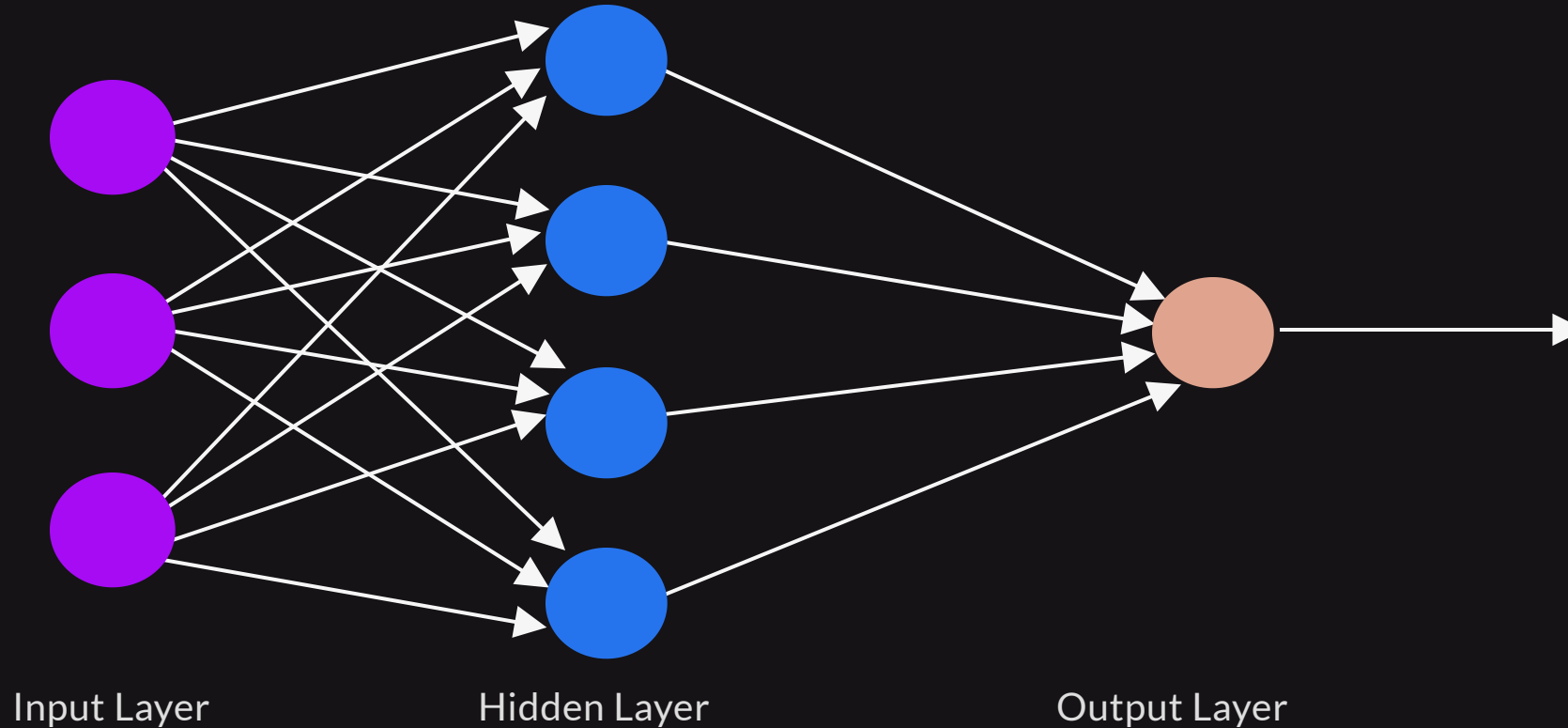
$$1 \leq x_1 + x_2 < 2$$



Types of Activation Functions

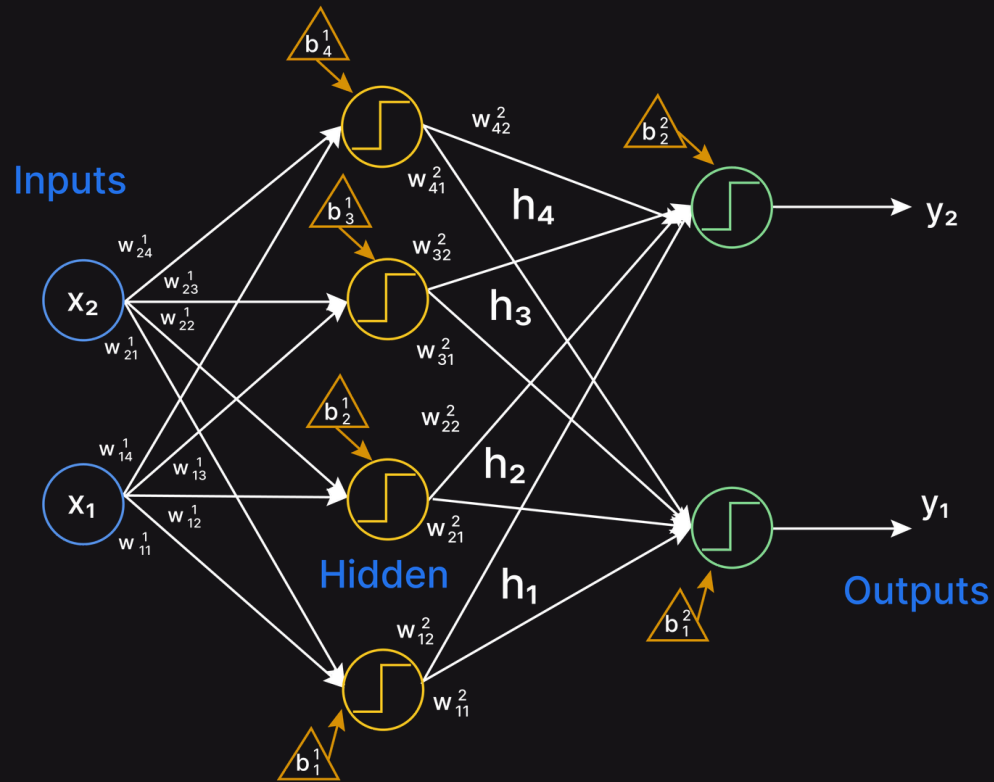


Universal Approximation Theorem



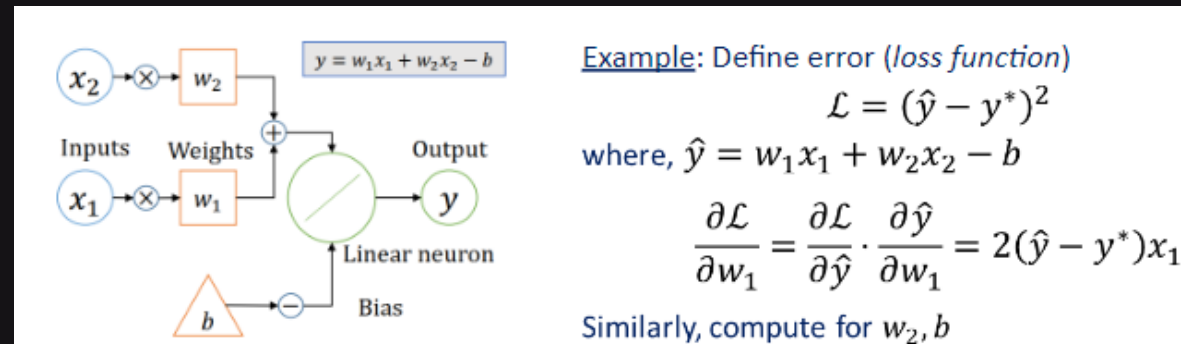
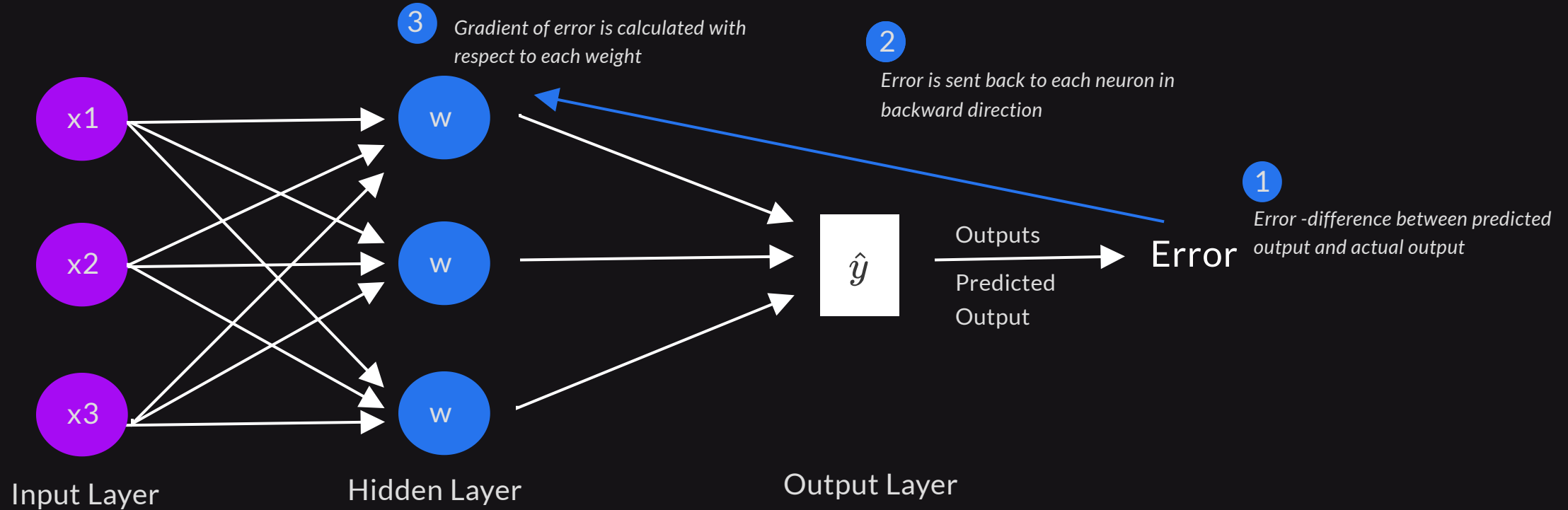
The universal approximation theorem states that any continuous function $f: [0, 1]^n \rightarrow [0, 1]$ can be approximated arbitrarily well by a neural network with at least 1 hidden layer with a finite number of weights, which is what we are going to illustrate in the next subsections.

Training Procedure: Backpropagation

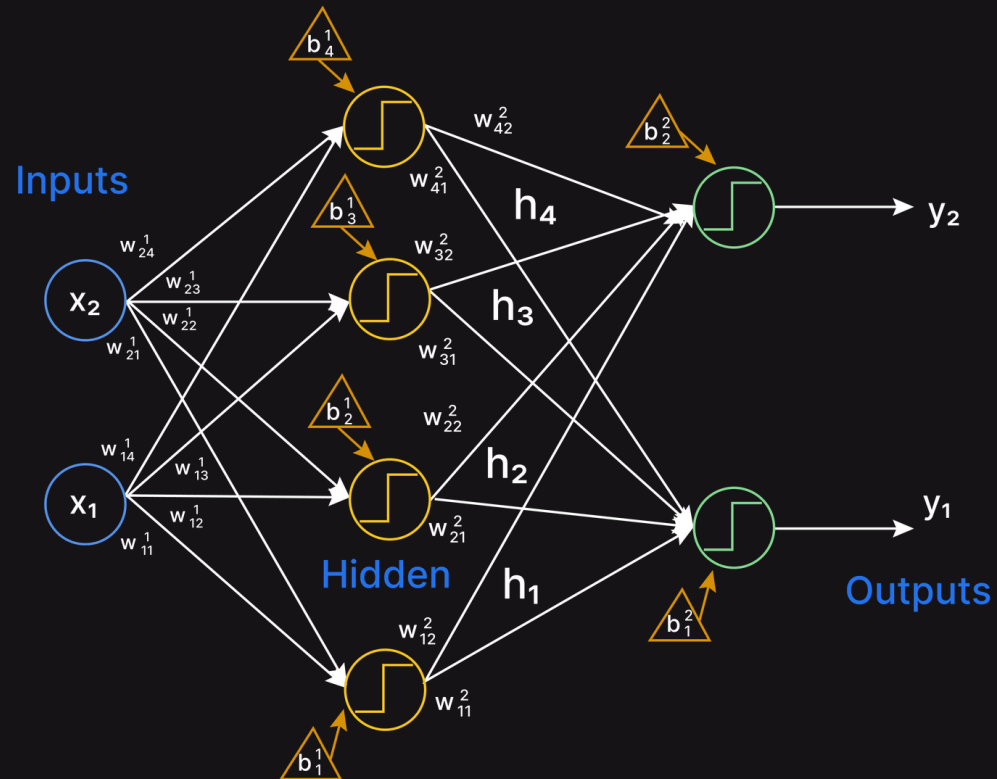


- With the right weights and biases, deeper networks learn more powerful concepts than shallow networks
- Problem: Computing huge numbers of weights and biases

Training Procedure: Backpropagation



Backpropagation for Neural Network



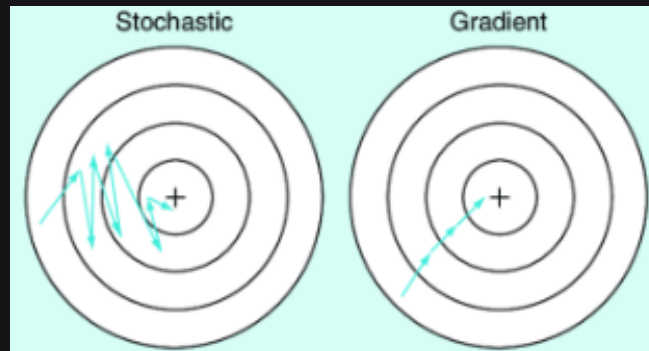
- Why: To compute 'correct' values of weights and biases
- Easy to do for single neurons (previously explained)
- Need chain rule for larger network
- Two steps involved in training process
 - Forward pass
 - Backward pass

Gradient Descent (GD)

- Update parameters using the following rule:

$$\theta_{t+1} = \theta_t - \frac{\eta}{k} \sum_{i=1}^k \nabla_{\theta} \mathcal{L}_i(\theta_t)$$

- We run this multiple times to find the right set of weights and biases
- Many variations exist: Adam, AdaGrad, RMSProp
- Sometimes , we regularize loss function to avoid potential overfitting



Thank You
